

Final Progress Report - Reinforcement Learning Agents for LunarLander

Work Completed

Environment setup and Baseline

The project uses Gymnasium's LunarLander-v3 environment with a discrete action space of four thruster commands and an 8-dimensional continuous state space. Initial setup includes `test_env.py` for verifying the environment's observation and action spaces, and `random_base_agent.py`, which runs a random-policy agent with human rendering to establish a qualitative baseline for task difficulty and typical failure modes. The random agent consistently scores between -150 and -300 per episode, crashing or drifting off-screen in nearly every attempt.

Deep Q-Network(DQN) Implementation

A complete DQN agent has been implemented in `dqn_agent.py` with the following components:

- **Q-Network Architecture:** Two fully connected hidden layers of 128 ReLU units, outputting Q-values for each of the four discrete actions. This architecture matches the reference study reviewed in the literature review to ensure controlled comparison.
- **Replay Buffer:** A circular buffer of capacity 100,000 that stores experience tuples and samples random mini-batches of size 64 to break temporal correlation during training.
- **Epsilon-Greedy Exploration:** Linear decay from 1.0 to 0.01 over 150,000 steps, balancing exploration and exploitation throughout training.
- **Target Network:** A separate copy of the Q-network, synchronized every 1,000 steps, to stabilize the TD target during updates.
- **Gradient Clipping:** Max norm of 1.0 to prevent training instability from exploding gradients.

The DQN agent was trained for 2,000 episodes across three random seeds (42, 100, 200) with a learning rate of $5e-4$ and discount factor of 0.99. Results show consistent convergence, with a best average reward of **256.24 ± 9.63** across seeds. All seeds exceeded a reward of 200, indicating successful landing behavior.

Proximal Policy Optimization (PPO) Implementation

A complete PPO agent has been implemented in `ppo_agent.py` with the following components:

- **Actor-Critic Network:** A shared two-layer backbone (128 ReLU units each) with separate actor (policy logits) and critic (state value) heads.

- **Generalized Advantage Estimation (GAE):** Computes advantages using $\lambda=0.95$ to balance bias and variance in the policy gradient estimate.
- **Clipped Surrogate Objective:** Constrains the policy update ratio within $[0.8, 1.2]$ to prevent destructively large policy changes in a single update.
- **Entropy Bonus:** Coefficient of 0.02 to encourage exploration and prevent premature convergence to suboptimal policies.
- **Mini-Batch Updates:** Each rollout of 2,048 steps is updated over 10 epochs with mini-batches of size 64.

The PPO agent was trained for 750,000 total timesteps across five random seeds (42, 100, 200, 300, 400) with a learning rate of $3e-4$. Results show strong and consistent performance, with a best average reward of **282.11 ± 6.92** across seeds. Every seed exceeded 270, demonstrating reliable convergence.

Visualization and Evaluation

Trained agents can be visually evaluated using `watch_trained_agent.py` (DQN) and `watch_trained_ppo.py` (PPO), which load saved model checkpoints and run the agent in the environment with human rendering. Both agents demonstrate qualitatively different behavior from the random baseline: they actively stabilize the lander, manage fuel usage, and execute controlled descents onto the landing pad.

Literature Review

A literature review was completed examining the study "Reinforcement Learning for Autonomous Lunar Landing: A Comparative Analysis of Algorithm Performance" (Subramaniam et al., 2025), which compares DQN variants and PPO on the same LunarLander environment. Key insights from this review — including multi-seed evaluation, consistent network architectures, and evaluation metrics — have been directly incorporated into the experimental design of this project.

Preliminary Results Summary

Metric	DQN	PPO
Best Avg Reward (mean ± std)	256.24 ± 9.63	282.11 ± 6.92
Number of Seeds	3	5
Peak Single-Seed Performance	269.81	291.28
Training Duration	2,000 episodes	750,000 steps (~2,254 episodes)

PPO achieved higher mean performance with lower variance, consistent with findings from the reviewed literature. DQN required more careful hyper-parameter tuning (two rounds of adjustment to learning rate and epsilon schedule) but still achieved strong results.

Remaining Work

Learning Curve Visualizations

Build a plotting script to generate reward-vs-episode learning curves for both DQN and PPO from the saved metrics JSON files. These plots will show mean reward with shaded standard deviation bands across seeds, providing a clear visual comparison of convergence speed and stability.

Extended Evaluation Metrics

Compute and report additional metrics from the saved training data: success rate (percentage of episodes with reward > 200), convergence speed (number of episodes to first reach a rolling average of 200), and average episode length as a proxy for fuel efficiency.

Final Paper

Write the complete project report including:

- Introduction and motivation for the problem
- Description of the methods (DQN and PPO) with algorithmic details
- Experimental setup and hyperparameters
- Results with learning curves and comparative analysis
- Discussion of why PPO outperformed DQN, behavioral observations (such as the hovering phase observed during DQN's epsilon transition), and connections to the reviewed literature
- - Conclusion and potential extensions (continuous action space, Dueling DQN)

Code Cleanup

Add inline comments, doc strings, and a complete README.md with setup instructions, dependency versions, and instructions for reproducing all experiments.